

Advanced DSA (Hard) – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 21 Dec 2026 – 3 Jan 2027 (Days 127–140)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](#)

Segment trees, tries, advanced DP, graph algorithms, monotonic stacks, 50 hard problems, templates, and 80 LPA interview preparation.

Target Level: Google L5, Meta E4, Amazon SDE3, Microsoft L63+ (80+ LPA TC in India)

PREREQUISITES – Complete these FIRST (do not skip)

Order	PDF	Why required
1	08b-dsa-datastructures-fundamentals-complete.pdf	Stack, heap, trie, union-find implementations
2	08-dsa-patterns-complete.pdf	Two pointers, DP, BFS/DFS patterns
3	200+ LeetCode mediums completed	Pattern recognition at speed

When to study: Month 5, Days 127–140 (after Week 19 trees/graphs).

Gate: Implement min-heap + reverse linked list from memory in 15 min.

Table of Contents

1. Segment Trees

2. Trie (Prefix Tree)
 3. Union-Find Advanced
 4. Topological Sort Variants
 5. Advanced Dynamic Programming
 6. Graph Algorithms Advanced
 7. Monotonic Stack & Queue Advanced
 8. 50 Hard LeetCode Problems
 9. NeetCode 150 + Hard Extension
 10. Interview Q&A (20 Questions)
 11. Where to Practice
-

1. Segment Trees

When to Use

- Range sum/min/max/query with point or range updates
- Problems requiring $O(\log n)$ range queries and updates
- Alternative to sparse table when updates are needed

Template – Range Sum with Point Update

```
class SegmentTree {
  private tree: number[];
  private n: number;

  constructor(arr: number[]) {
    this.n = arr.length;
    this.tree = new Array(4 * this.n).fill(0);
    this.build(arr, 0, 0, this.n - 1);
  }

  private build(arr: number[], node: number, start: number, end: number): void {
    if (start === end) {
      this.tree[node] = arr[start];
      return;
    }
    const mid = Math.floor((start + end) / 2);
    this.build(arr, 2 * node + 1, start, mid);
    this.build(arr, 2 * node + 2, mid + 1, end);
    this.tree[node] = this.tree[2 * node + 1] + this.tree[2 * node + 2];
  }

  update(index: number, val: number): void {
    this.updateTree(0, 0, this.n - 1, index, val);
  }

  private updateTree(node: number, start: number, end: number, idx: number, val: number): void {
    if (start === end) {
      this.tree[node] = val;
      return;
    }
    const mid = Math.floor((start + end) / 2);
    if (idx <= mid) {
      this.updateTree(2 * node + 1, start, mid, idx, val);
    } else {
      this.updateTree(2 * node + 2, mid + 1, end, idx, val);
    }
    this.tree[node] = this.tree[2 * node + 1] + this.tree[2 * node + 2];
  }

  query(left: number, right: number): number {
    return this.queryTree(0, 0, this.n - 1, left, right);
  }

  private queryTree(node: number, start: number, end: number, left: number, right: number): number {
    if (right < start || end < left) return 0;
    if (left <= start && end <= right) return this.tree[node];
    const mid = Math.floor((start + end) / 2);
    return this.queryTree(2 * node + 1, start, mid, left, right) +
```

```
        this.queryTree(2 * node + 2, mid + 1, end, left, right);  
    }  
}
```

Time: Build $O(n)$, Query $O(\log n)$, Update $O(\log n)$ | **Space:** $O(n)$

Lazy Propagation (Range Updates)

For range update + range query, use lazy propagation to defer updates.

```
class LazySegmentTree {
  private tree: number[];
  private lazy: number[];
  private n: number;

  constructor(arr: number[]) {
    this.n = arr.length;
    this.tree = new Array(4 * this.n).fill(0);
    this.lazy = new Array(4 * this.n).fill(0);
    this.build(arr, 0, 0, this.n - 1);
  }

  private build(arr: number[], node: number, start: number, end: number): void {
    if (start === end) { this.tree[node] = arr[start]; return; }
    const mid = Math.floor((start + end) / 2);
    this.build(arr, 2 * node + 1, start, mid);
    this.build(arr, 2 * node + 2, mid + 1, end);
    this.tree[node] = this.tree[2 * node + 1] + this.tree[2 * node + 2];
  }

  private pushDown(node: number, start: number, end: number): void {
    if (this.lazy[node] !== 0) {
      this.tree[node] += this.lazy[node] * (end - start + 1);
      if (start !== end) {
        this.lazy[2 * node + 1] += this.lazy[node];
        this.lazy[2 * node + 2] += this.lazy[node];
      }
      this.lazy[node] = 0;
    }
  }

  rangeUpdate(left: number, right: number, val: number): void {
    this.updateRange(0, 0, this.n - 1, left, right, val);
  }

  private updateRange(node: number, start: number, end: number, left: number, right: number, val: number): void {
    this.pushDown(node, start, end);
    if (right < start || end < left) return;
    if (left <= start && end <= right) {
      this.lazy[node] += val;
      this.pushDown(node, start, end);
      return;
    }
    const mid = Math.floor((start + end) / 2);
    this.updateRange(2 * node + 1, start, mid, left, right, val);
    this.updateRange(2 * node + 2, mid + 1, end, left, right, val);
    this.pushDown(2 * node + 1, start, mid);
    this.pushDown(2 * node + 2, mid + 1, end);
    this.tree[node] = this.tree[2 * node + 1] + this.tree[2 * node + 2];
  }
}
```

```
rangeQuery(left: number, right: number): number {
    return this.queryRange(0, 0, this.n - 1, left, right);
}

private queryRange(node: number, start: number, end: number, left: number, right: number): number {
    this.pushDown(node, start, end);
    if (right < start || end < left) return 0;
    if (left <= start && end <= right) return this.tree[node];
    const mid = Math.floor((start + end) / 2);
    return this.queryRange(2 * node + 1, start, mid, left, right) +
        this.queryRange(2 * node + 2, mid + 1, end, left, right);
}
```

Time: $O(\log n)$ per operation | **Space:** $O(n)$

Key Problems: LC 307, 315, 327, 699, 729

2. Trie (Prefix Tree)

Template

```
class TrieNode {
  children: Map<string, TrieNode> = new Map();
  isEnd: boolean = false;
  count: number = 0; // For prefix counting
}

class Trie {
  private root = new TrieNode();

  insert(word: string): void {
    let node = this.root;
    for (const char of word) {
      if (!node.children.has(char)) {
        node.children.set(char, new TrieNode());
      }
      node = node.children.get(char)!;
      node.count++;
    }
    node.isEnd = true;
  }

  search(word: string): boolean {
    const node = this.traverse(word);
    return node !== null && node.isEnd;
  }

  startsWith(prefix: string): boolean {
    return this.traverse(prefix) !== null;
  }

  countPrefix(prefix: string): number {
    const node = this.traverse(prefix);
    return node ? node.count : 0;
  }

  private traverse(str: string): TrieNode | null {
    let node = this.root;
    for (const char of str) {
      if (!node.children.has(char)) return null;
      node = node.children.get(char)!;
    }
    return node;
  }
}
```

```
}  
}
```

Time: Insert/Search/Prefix $O(m)$ where m = word length | **Space:** $O(n \times m)$

Advanced: Binary Trie (for Bit Manipulation)

Used for maximum XOR queries (LC 421, 1707).

```
class BinaryTrieNode {  
    children: [BinaryTrieNode | null, BinaryTrieNode | null] = [null, null];  
}  
  
class BinaryTrie {  
    private root = new BinaryTrieNode();  
  
    insert(num: number): void {  
        let node = this.root;  
        for (let i = 31; i >= 0; i--) {  
            const bit = (num >> i) & 1;  
            if (!node.children[bit]) {  
                node.children[bit] = new BinaryTrieNode();  
            }  
            node = node.children[bit]!;  
        }  
    }  
  
    findMaxXOR(num: number): number {  
        let node = this.root;  
        let xor = 0;  
        for (let i = 31; i >= 0; i--) {  
            const bit = (num >> i) & 1;  
            const want = 1 - bit; // Greedy: pick opposite bit for max XOR  
            if (node.children[want]) {  
                xor |= (1 << i);  
                node = node.children[want]!;  
            } else {  
                node = node.children[bit]!;  
            }  
        }  
        return xor;  
    }  
}
```

Time: $O(32) = O(1)$ per operation | **Space:** $O(n \times 32)$

Key Problems: LC 208, 211, 212, 421, 648, 1268, 1707

3. Union-Find Advanced

Template with Path Compression + Union by Rank

```
class UnionFind {
  private parent: number[];
  private rank: number[];
  private count: number;

  constructor(n: number) {
    this.parent = Array.from({ length: n }, (_, i) => i);
    this.rank = new Array(n).fill(0);
    this.count = n;
  }

  find(x: number): number {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]); // Path compression
    }
    return this.parent[x];
  }

  union(x: number, y: number): boolean {
    const px = this.find(x);
    const py = this.find(y);
    if (px === py) return false;

    // Union by rank
    if (this.rank[px] < this.rank[py]) {
      this.parent[px] = py;
    } else if (this.rank[px] > this.rank[py]) {
      this.parent[py] = px;
    } else {
      this.parent[py] = px;
      this.rank[px]++;
    }
    this.count--;
    return true;
  }

  connected(x: number, y: number): boolean {
    return this.find(x) === this.find(y);
  }

  getCount(): number {
    return this.count;
  }
}
```

```
}  
}
```

Time: $O(\alpha(n)) \approx O(1)$ amortized per operation | **Space:** $O(n)$

Union-Find with Extra State

```
// Track component sizes  
class UnionFindWithSize {  
  private parent: number[];  
  private size: number[];  
  
  constructor(n: number) {  
    this.parent = Array.from({ length: n }, (_, i) => i);  
    this.size = new Array(n).fill(1);  
  }  
  
  find(x: number): number {  
    if (this.parent[x] !== x) this.parent[x] = this.find(this.parent[x]);  
    return this.parent[x];  
  }  
  
  union(x: number, y: number): number {  
    const px = this.find(x), py = this.find(y);  
    if (px === py) return this.size[px];  
    if (this.size[px] < this.size[py]) {  
      this.parent[px] = py;  
      this.size[py] += this.size[px];  
      return this.size[py];  
    } else {  
      this.parent[py] = px;  
      this.size[px] += this.size[py];  
      return this.size[px];  
    }  
  }  
}
```

Key Problems: LC 200, 305, 547, 684, 721, 947, 1202, 1319, 1697

4. Topological Sort Variants

Kahn's Algorithm (BFS)

```
function topologicalSortBFS(numNodes: number, edges: number[][]): number[] {
  const graph = new Map<number, number[]>();
  const inDegree = new Array(numNodes).fill(0);

  for (const [u, v] of edges) {
    if (!graph.has(u)) graph.set(u, []);
    graph.get(u)!.push(v);
    inDegree[v]++;
  }

  const queue: number[] = [];
  for (let i = 0; i < numNodes; i++) {
    if (inDegree[i] === 0) queue.push(i);
  }

  const order: number[] = [];
  while (queue.length > 0) {
    const node = queue.shift()!;
    order.push(node);
    for (const neighbor of graph.get(node) || []) {
      inDegree[neighbor]--;
      if (inDegree[neighbor] === 0) queue.push(neighbor);
    }
  }

  return order.length === numNodes ? order : []; // Empty = cycle detected
}
```

Time: $O(V + E)$ | **Space:** $O(V + E)$

DFS-Based Topological Sort

```
function topologicalSortDFS(numNodes: number, edges: number[][]): number[] {
  const graph = new Map<number, number[]>();
  for (const [u, v] of edges) {
    if (!graph.has(u)) graph.set(u, []);
    graph.get(u)!.push(v);
  }

  const visited = new Set<number>();
  const inStack = new Set<number>();
  const order: number[] = [];

  function dfs(node: number): boolean {
    visited.add(node);
    inStack.add(node);

    for (const neighbor of graph.get(node) || []) {
      if (inStack.has(neighbor)) return false; // Cycle
      if (!visited.has(neighbor) && !dfs(neighbor)) return false;
    }

    inStack.delete(node);
    order.push(node);
    return true;
  }

  for (let i = 0; i < numNodes; i++) {
    if (!visited.has(i) && !dfs(i)) return [];
  }

  return order.reverse();
}
```

Lexicographically Smallest Topological Order

Use min-heap instead of queue in Kahn's algorithm (LC 1857).

```
function lexSmallestTopoSort(n: number, edges: number[][]): number[] {
  const graph = new Map<number, number[]>();
  const inDegree = new Array(n).fill(0);

  for (const [u, v] of edges) {
    if (!graph.has(u)) graph.set(u, []);
    graph.get(u)!.push(v);
    inDegree[v]++;
  }

  // Min heap (using sorted array for simplicity)
  const heap: number[] = [];
  for (let i = 0; i < n; i++) if (inDegree[i] === 0) heap.push(i);
  heap.sort((a, b) => b - a); // Reverse for pop-min via pop()

  const order: number[] = [];
  while (heap.length > 0) {
    const node = heap.pop();
    order.push(node);
    for (const neighbor of graph.get(node) || []) {
      inDegree[neighbor]--;
      if (inDegree[neighbor] === 0) {
        heap.push(neighbor);
        heap.sort((a, b) => b - a);
      }
    }
  }

  return order.length === n ? order : [];
}
```

Key Problems: LC 207, 210, 269, 310, 329, 802, 1203, 1857

5. Advanced Dynamic Programming

5.1 Bitmask DP

Use when: Small n (≤ 20), need to track subset states.

Template — Traveling Salesman Problem (TSP):

```
function tsp(dist: number[][]): number {
  const n = dist.length;
  const ALL = (1 << n) - 1;
  const dp: number[][] = Array.from({ length: 1 << n }, () => new Array(n).fill(Infinity));
  dp[1][0] = 0; // Start at city 0

  for (let mask = 1; mask <= ALL; mask++) {
    for (let u = 0; u < n; u++) {
      if (!(mask & (1 << u))) continue;
      if (dp[mask][u] === Infinity) continue;

      for (let v = 0; v < n; v++) {
        if (mask & (1 << v)) continue;
        const newMask = mask | (1 << v);
        dp[newMask][v] = Math.min(dp[newMask][v], dp[mask][u] + dist[u][v]);
      }
    }
  }

  let result = Infinity;
  for (let u = 1; u < n; u++) {
    result = Math.min(result, dp[ALL][u] + dist[u][0]);
  }
  return result;
}
```

Time: $O(n^2 \times 2^n)$ | **Space:** $O(n \times 2^n)$

Key Problems: LC 847, 943, 996, 1125, 1349, 1434, 1723

5.2 Digit DP

Use when: Count numbers in range $[L, R]$ satisfying digit constraints.

Template — Count numbers with digit sum = target:

```
function countNums(s: string, minSum: number, maxSum: number): number {
  const MOD = 1e9 + 7;
  const n = s.length;
  const memo = new Map<string, number>();

  function dp(pos: number, sum: number, tight: boolean): number {
    if (pos === n) return sum >= minSum && sum <= maxSum ? 1 : 0;
    const key = `${pos},${sum},${tight}`;
    if (memo.has(key)) return memo.get(key)!;

    const limit = tight ? parseInt(s[pos]) : 9;
    let result = 0;
    for (let d = 0; d <= limit; d++) {
      result = (result + dp(pos + 1, sum + d, tight && d === limit)) % MOD;
    }
    memo.set(key, result);
    return result;
  }

  return dp(0, 0, true);
}
```

Time: $O(n \times \text{sum} \times 10)$ | **Space:** $O(n \times \text{sum})$

Key Problems: LC 233, 357, 600, 902, 1012, 1067

5.3 Tree DP

Template — House Robber III (include/exclude at each node):

```
interface RobResult {
  withRoot: number;
  withoutRoot: number;
}

function rob(root: TreeNode | null): number {
  function dp(node: TreeNode | null): RobResult {
    if (!node) return { withRoot: 0, withoutRoot: 0 };

    const left = dp(node.left);
    const right = dp(node.right);

    return {
      withRoot: node.val + left.withoutRoot + right.withoutRoot,
      withoutRoot: Math.max(left.withRoot, left.withoutRoot) +
        Math.max(right.withRoot, right.withoutRoot),
    };
  }

  const result = dp(root);
  return Math.max(result.withRoot, result.withoutRoot);
}
```

Time: $O(n)$ | Space: $O(h)$ recursion stack

Key Problems: LC 337, 124, 968, 979, 1377, 2246

5.4 DP on Graphs

Template — Shortest Path with exactly K edges:

```
function shortestPathWithKEdges(n: number, edges: number[][], src: number, dst: number) {
  const INF = Infinity;
  let dp = new Array(n).fill(INF);
  dp[src] = 0;

  for (let i = 0; i <= k; i++) {
    const next = [...dp];
    for (const [u, v, w] of edges) {
      if (dp[u] !== INF) {
        next[v] = Math.min(next[v], dp[u] + w);
      }
    }
    dp = next;
  }

  return dp[dst] === INF ? -1 : dp[dst];
}
```

Key Problems: LC 787, 1334, 1462, 1697

6. Graph Algorithms Advanced

6.1 Dijkstra's Algorithm

```
function dijkstra(graph: Map<number, [number, number][]>, start: number, n: number) {
  const dist = new Array(n).fill(Infinity);
  dist[start] = 0;

  // Min heap: [distance, node]
  const heap: [number, number][] = [[0, start]];

  while (heap.length > 0) {
    heap.sort((a, b) => b[0] - a[0]); // Pop min (simple impl)
    const [d, u] = heap.pop()!;

    if (d > dist[u]) continue;

    for (const [v, w] of graph.get(u) || []) {
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        heap.push([dist[v], v]);
      }
    }
  }

  return dist;
}
```

Time: $O((V + E) \log V)$ with binary heap | **Space:** $O(V)$

Key Problems: LC 743, 787, 1514, 1631, 1786, 1976

6.2 Bellman-Ford

Handles negative weights, detects negative cycles.

```
function bellmanFord(n: number, edges: number[][], src: number): number[] | null {
  const dist = new Array(n).fill(Infinity);
  dist[src] = 0;

  for (let i = 0; i < n - 1; i++) {
    for (const [u, v, w] of edges) {
      if (dist[u] !== Infinity && dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
      }
    }
  }

  // Check for negative cycle
  for (const [u, v, w] of edges) {
    if (dist[u] !== Infinity && dist[u] + w < dist[v]) {
      return null; // Negative cycle
    }
  }

  return dist;
}
```

Time: $O(V \times E)$ | Space: $O(V)$

Key Problems: LC 787, 994, 1334, 1462

6.3 Floyd-Warshall (All-Pairs Shortest Path)

```
function floydWarshall(dist: number[][]): number[][] {
  const n = dist.length;

  for (let k = 0; k < n; k++) {
    for (let i = 0; i < n; i++) {
      for (let j = 0; j < n; j++) {
        if (dist[i][k] !== Infinity && dist[k][j] !== Infinity) {
          dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
        }
      }
    }
  }

  return dist;
}
```

Time: $O(V^3)$ | Space: $O(V^2)$

Key Problems: LC 1334, 1462, 2045

6.4 Minimum Spanning Tree

Kruskal's (Union-Find):

```
function kruskal(n: number, edges: number[][]): number {
  edges.sort((a, b) => a[2] - b[2]); // Sort by weight
  const uf = new UnionFind(n);
  let cost = 0;
  let edgesUsed = 0;

  for (const [u, v, w] of edges) {
    if (uf.union(u, v)) {
      cost += w;
      edgesUsed++;
      if (edgesUsed === n - 1) break;
    }
  }

  return edgesUsed === n - 1 ? cost : -1;
}
```

Prim's (Min Heap):

```
function prim(n: number, graph: Map<number, [number, number][]>): number {
  const visited = new Set<number>();
  const heap: [number, number][] = [[0, 0]]; // [weight, node]
  let cost = 0;

  while (heap.length > 0 && visited.size < n) {
    heap.sort((a, b) => b[0] - a[0]);
    const [w, u] = heap.pop()!;
    if (visited.has(u)) continue;
    visited.add(u);
    cost += w;

    for (const [v, weight] of graph.get(u) || []) {
      if (!visited.has(v)) heap.push([weight, v]);
    }
  }

  return visited.size === n ? cost : -1;
}
```

Time: Kruskal $O(E \log E)$, Prim $O(E \log V)$ | **Space:** $O(V + E)$

Key Problems: LC 1135, 1168, 1489, 1584, 1724

7. Monotonic Stack & Queue Advanced

Monotonic Stack Template

Next Greater Element:

```
function nextGreaterElements(nums: number[]): number[] {
  const n = nums.length;
  const result = new Array(n).fill(-1);
  const stack: number[] = []; // Indices, decreasing values

  for (let i = 0; i < 2 * n; i++) {
    const idx = i % n;
    while (stack.length > 0 && nums[stack[stack.length - 1]] < nums[idx]) {
      result[stack.pop()!] = nums[idx];
    }
    if (i < n) stack.push(idx);
  }

  return result;
}
```

Time: $O(n)$ | **Space:** $O(n)$

Largest Rectangle in Histogram

```
function largestRectangleArea(heights: number[]): number {
  const stack: number[] = []; // Increasing indices
  let maxArea = 0;

  for (let i = 0; i <= heights.length; i++) {
    const h = i === heights.length ? 0 : heights[i];
    while (stack.length > 0 && heights[stack[stack.length - 1]] > h) {
      const height = heights[stack.pop()!];
      const width = stack.length === 0 ? i : i - stack[stack.length - 1] - 1;
      maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
  }

  return maxArea;
}
```

Time: $O(n)$ | Space: $O(n)$

Monotonic Deque (Sliding Window Maximum)

```
function maxSlidingWindow(nums: number[], k: number): number[] {
  const deque: number[] = []; // Indices, decreasing values
  const result: number[] = [];

  for (let i = 0; i < nums.length; i++) {
    // Remove out-of-window indices
    while (deque.length > 0 && deque[0] <= i - k) {
      deque.shift();
    }
    // Maintain decreasing order
    while (deque.length > 0 && nums[deque[deque.length - 1]] <= nums[i]) {
      deque.pop();
    }
    deque.push(i);

    if (i >= k - 1) result.push(nums[deque[0]]);
  }

  return result;
}
```

Time: $O(n)$ | **Space:** $O(k)$

Key Problems: LC 42, 84, 85, 239, 316, 402, 503, 739, 840, 862, 907, 2104

8. 50 Hard LeetCode Problems

#	Problem	Pattern	Approach Summary	Time	Space
1	4 (Median of Two Sorted Arrays)	Binary Search	Binary search on partition point in smaller array	$O(\log(\min(m,n)))$	$O(1)$
2	10 (Regular Expression Matching)	DP	2D DP: $dp[i][j]$ = match $s[0..i]$ with $p[0..j]$	$O(mn)$	$O(mn)$
3	23 (Merge k Sorted Lists)	Heap	Min heap of k list heads, pop and push next	$O(n \log k)$	$O(k)$
4	25 (Reverse Nodes in k-Group)	Linked List	Reverse k nodes at a time, handle remaining	$O(n)$	$O(1)$
5	30 (Substring with Concatenation)	Sliding Window	Fixed window of word length $\times$ num words	$O(n \times m)$	$O(m)$
6	32 (Longest Valid Parentheses)	Stack/DP	Stack tracks indices, calculate lengths	$O(n)$	$O(n)$
7	37 (Sudoku Solver)	Backtracking	Try digits 1-9, backtrack on conflict	$O(9^{(n \times n)})$	$O(n^2)$
8	42 (Trapping Rain Water)	Monotonic Stack	Two pointers or stack for bounded water	$O(n)$	$O(1)$
9	44 (Wildcard Matching)	DP	Similar to regex but '?' and '*' only	$O(mn)$	$O(mn)$

10	51 (N-Queens)	Backtracking	Place queen row by row, check cols/diagonals	$O(n!)$	$O(n^2)$
11	72 (Edit Distance)	DP	Insert/delete/replace operations	$O(mn)$	$O(mn)$
12	76 (Minimum Window Substring)	Sliding Window	Expand/shrink window, track char counts	$O(n)$	$O(k)$
13	84 (Largest Rectangle in Histogram)	Monotonic Stack	Stack maintains increasing heights	$O(n)$	$O(n)$
14	85 (Maximal Rectangle)	Monotonic Stack	Apply histogram to each row of matrix	$O(mn)$	$O(n)$
15	87 (Scramble String)	DP	$dp[i][j][k] = \text{is } s1[i..i+k] \text{ scramble of } s2[j..j+k]$	$O(n^4)$	$O(n^3)$
16	115 (Distinct Subsequences)	DP	Count t subsequences in s	$O(mn)$	$O(mn)$
17	123 (Best Time III)	DP	$dp[k][\text{day}] = \text{max profit with } k \text{ transactions}$	$O(kn)$	$O(kn)$
18	124 (Binary Tree Max Path Sum)	Tree DP	Include/exclude each node in path	$O(n)$	$O(h)$
19	126 (Word Ladder II)	BFS + Backtracking	BFS for shortest, backtrack all paths	$O(n \times 26^L)$	$O(n)$
20	127 (Word Ladder)	BFS	Bidirectional BFS for shortest transformation	$O(n \times 26^L)$	$O(n)$

21	128 (Longest Consecutive)	Union-Find/HashSet	Track consecutive sequences	$O(n)$	$O(n)$
22	132 (Palindrome Partitioning II)	DP	Min cuts for palindrome partition	$O(n^2)$	$O(n^2)$
23	135 (Candy)	Greedy	Two passes: left to right, right to left	$O(n)$	$O(n)$
24	140 (Word Break II)	DP + Backtracking	$dp[i]$ = all sentences for $s[0..i]$	$O(n^2 \times 2^n)$	$O(n^3)$
25	149 (Max Points on Line)	Math/Hash	Slope counting with GCD normalization	$O(n^2)$	$O(n)$
26	154 (Find Minimum Rotated II)	Binary Search	Handle duplicates with $left++$	$O(\log n)$	$O(1)$
27	188 (Best Time IV)	DP	Generalized k transactions DP	$O(kn)$	$O(kn)$
28	212 (Word Search II)	Trie + Backtracking	Build trie from words, DFS on board	$O(mn \times 4^L)$	$O(wL)$
29	214 (Shortest Palindrome)	KMP/Rolling Hash	Find longest palindromic prefix	$O(n)$	$O(n)$
30	218 (Skyline Problem)	Sweep Line/Heap	Process events sorted by x coordinate	$O(n \log n)$	$O(n)$
31	224 (Basic Calculator)	Stack	Handle parentheses and operators	$O(n)$	$O(n)$
32	239 (Sliding Window Maximum)	Monotonic Deque	Deque maintains decreasing values	$O(n)$	$O(k)$

33	295 (Find Median Stream)	Two Heaps	Max heap (lower) + min heap (upper)	$O(\log n)$ insert	$O(n)$
34	297 (Serialize BT)	Tree/BFS	BFS or preorder with null markers	$O(n)$	$O(n)$
35	312 (Burst Balloons)	Interval DP	$dp[i][j] = \max$ coins bursting range $i..j$ last	$O(n^3)$	$O(n^2)$
36	315 (Count Smaller After Self)	Merge Sort/BST	Count inversions during merge sort	$O(n \log n)$	$O(n)$
37	329 (Longest Increasing Path)	DFS + Memo	Top-down DP on grid with memoization	$O(mn)$	$O(mn)$
38	332 (Reconstruct Itinerary)	Eulerian Path	Hierholzer's algorithm on graph	$O(E \log E)$	$O(E)$
39	335 (Self Crossing)	Geometry	Analyze line segment intersections	$O(n)$	$O(1)$
40	354 (Russian Doll Envelopes)	DP + LIS	Sort by width, LIS on heights	$O(n \log n)$	$O(n)$
41	403 (Frog Jump)	DP	$dp[\text{stone}] = \text{set}$ of possible jump sizes	$O(n^2)$	$O(n^2)$
42	410 (Split Array Largest Sum)	Binary Search	Binary search on answer (min max sum)	$O(n \log S)$	$O(1)$
43	421 (Maximum XOR)	Binary Trie	Insert nums, greedily pick opposite bits	$O(n \times 32)$	$O(n \times 32)$
44	446 (Arithmetic Slices II)	DP	$dp[\text{diff}][\text{end}] = \text{count}$ of sequences	$O(n^2)$	$O(n^2)$

45	460 (LFU Cache)	Design	HashMap + freq buckets + doubly linked list	$O(1)$ ops	$O(\text{cap})$
46	472 (Concatenated Words)	Trie + DP	$\text{dp}[i]$ = can $s[0..i]$ be formed from words	$O(n^2 \times m)$	$O(n^2)$
47	493 (Reverse Pairs)	Merge Sort	Count pairs during merge sort	$O(n \log n)$	$O(n)$
48	502 (IPO)	Greedy + Heap	Max heap for profits, min heap for capital	$O(n \log n)$	$O(n)$
49	664 (Strange Printer)	Interval DP	$\text{dp}[i][j]$ = min turns to print $s[i..j]$	$O(n^3)$	$O(n^2)$
50	871 (Min Refuel Stops)	DP/Greedy	$\text{dp}[i]$ = max distance with i stops	$O(n^2)$	$O(n)$

9. NeetCode 150 + Hard Extension

NeetCode 150 – Hard Problems to Prioritize

Category	Must-Do Hard Problems
Arrays	4, 42, 84, 239, 295
Two Pointers	76, 84
Sliding Window	76, 239
Stack	42, 84, 85
Binary Search	4, 154, 410
Linked List	23, 25
Trees	124, 297
Tries	212
Backtracking	51, 37
Heap	23, 295
Graphs	127, 332
DP 1D	72, 132, 139, 140
DP 2D	72, 115, 312
Greedy	135, 354
Intervals	218
Math	149

Hard Extension List (Beyond NeetCode 150)

Problem	Why Important
LC 10 (Regex Matching)	Classic 2D DP, FAANG favorite
LC 44 (Wildcard Matching)	Variant of regex DP
LC 87 (Scramble String)	3D DP, interval DP
LC 115 (Distinct Subsequences)	String DP pattern
LC 123/188 (Stock variants)	State machine DP
LC 315 (Count Smaller)	Merge sort / BIT
LC 327 (Count Range Sum)	Merge sort / BIT
LC 403 (Frog Jump)	Set-based DP
LC 446 (Arithmetic Slices II)	Map-based DP
LC 493 (Reverse Pairs)	Merge sort inversion count
LC 664 (Strange Printer)	Interval DP
LC 871 (Min Refuel Stops)	DP / greedy with heap
LC 902 (Numbers At Most N)	Digit DP
LC 968 (Binary Tree Cameras)	Tree DP with states
LC 996 (Squareful Arrays)	Bitmask DP
LC 1125 (Smallest Sufficient Team)	Bitmask DP
LC 1434 (Max Points in Grid)	Bitmask DP
LC 1723 (Min Time to Finish Jobs)	Bitmask DP
LC 1857 (Largest Color Value)	Topo sort + DP
LC 2104 (Sum of Subarray Ranges)	Monotonic stack

10. Interview Q&A (20 Questions)

Q1: Explain segment tree vs binary indexed tree (Fenwick tree). When would you use each?

Answer: Both support range queries and point updates in $O(\log n)$, but differ in flexibility and implementation complexity.

A **Binary Indexed Tree (Fenwick tree)** is simpler to implement (~ 10 lines of code), uses $O(n)$ space, and supports prefix sum queries and point updates. It can handle range sum queries with two prefix queries. However, it only works for invertible operations (sum, XOR) and doesn't natively support range updates or non-invertible operations like min/max range queries.

A **Segment tree** is more flexible — it supports any associative operation (sum, min, max, GCD, XOR) with both point and range updates via lazy propagation. It uses $O(4n)$ space and is more complex to implement ($\sim 50-80$ lines). Segment trees can answer range min/max queries that Fenwick trees cannot.

Use Fenwick tree when: You need prefix sums with point updates, implementation speed matters (competitive programming), and memory is tight. **Use segment tree when:** You need range min/max/GCD, range updates, or non-invertible operations. In FAANG interviews, if the problem asks for range minimum query with updates, segment tree is the answer. If it's range sum query with point updates, either works but Fenwick tree shows you know the optimal approach.

Q2: How does bitmask DP work? Give an example.

Answer: Bitmask DP uses integers as bitmasks to represent subsets, enabling DP over exponential state spaces in $O(n \times 2^n)$ instead of $O(n!)$.

Each bit in the mask represents whether an element is included: `mask = 0b1011` means elements 0, 1, and 3 are included. We iterate over all 2^n masks and compute transitions by adding/removing one element.

Classic example — Traveling Salesman Problem: State `dp[mask][i]` = minimum cost to visit all cities in `mask`, ending at city `i`. Transition: `dp[mask | (1<<j)][j] = min(dp[mask | (1<<j)][j], dp[mask][i] + dist[i][j])` for all `j` not in `mask`.

Another example — Assignment Problem (LC 1879): Assign `n` workers to `n` jobs, minimize cost. State `dp[mask]` = minimum cost to assign jobs represented by `mask`. Transition: try assigning each

unassigned job to the next worker.

Implementation tips: Use `(1 << n) - 1` for the "all elements" mask. Iterate masks in increasing order so submasks are already computed. Use `mask & (1 << i)` to check if element `i` is included. Prune with bounds when possible.

When to recognize: Problem says $n \leq 20$, involves subsets/permutations/assignments, and brute force $O(n!)$ is too slow. Time complexity is always $O(n^2 \times 2^n)$ or $O(n \times 2^n)$.

Q3: Explain the monotonic stack pattern and its applications.

Answer: A monotonic stack maintains elements in sorted order (increasing or decreasing). When a new element violates the order, we pop elements and process them — this reveals useful information about the popped elements.

Core insight: When we pop element at index `i` because we found a smaller element at index `j`, we know that `j` is the **next smaller element** to the right of `i`. The width of the rectangle (for histogram problems) is `j - stack.top() - 1`.

Key applications:

1. **Next Greater/Smaller Element (LC 503, 739):** For each element, find the next greater element to its right. Maintain decreasing stack; when current element is greater than stack top, stack top's next greater is current element.
2. **Largest Rectangle in Histogram (LC 84):** Maintain increasing stack of indices. When height decreases, calculate rectangle area for popped index. Width = current index - new stack top - 1.
3. **Trapping Rain Water (LC 42):** Can use monotonic stack or two pointers. Stack approach: when current bar is taller than stack top, water is trapped between them.
4. **Daily Temperatures (LC 739):** Classic next greater element. $O(n)$ because each element is pushed and popped at most once.
5. **Sum of Subarray Minimums (LC 907):** For each element, find previous and next smaller elements using monotonic stack, calculate contribution.

Time complexity: Always $O(n)$ — each element is pushed and popped at most once. **Space:** $O(n)$ for the stack. In interviews, explain WHY it's $O(n)$ — the amortized analysis of push/pop operations.

Q4: How do you solve the Word Ladder problem (LC 127)?

Answer: Word Ladder finds the shortest transformation sequence from beginWord to endWord, changing one letter at a time, using only words from the word list. This is a shortest path problem → BFS.

Standard BFS approach:

1. Build a set from wordList for $O(1)$ lookup
2. BFS from beginWord, each level = one transformation
3. For each word, try changing each character to a-z, check if result is in wordList
4. Track visited words to avoid cycles
5. Return level when endWord is reached, or 0 if impossible

Time: $O(n \times m \times 26)$ where n = word list size, m = word length. **Space:** $O(n)$

Optimized — Bidirectional BFS: Search simultaneously from beginWord (forward) and endWord (backward). When the two frontiers meet, return combined depth. This reduces search space from $O(b^d)$ to $O(b^{(d/2)})$ — dramatic speedup for long chains.

```
function ladderLength(beginWord: string, endWord: string, wordList: string[]): number {
    const wordSet = new Set(wordList);
    if (!wordSet.has(endWord)) return 0;

    let forward = new Set([beginWord]);
    let backward = new Set([endWord]);
    let level = 1;

    while (forward.size > 0 && backward.size > 0) {
        // Always expand smaller set
        if (forward.size > backward.size) [forward, backward] = [backward, forward];

        const nextLevel = new Set<string>();
        for (const word of forward) {
            for (let i = 0; i < word.length; i++) {
                for (let c = 97; c <= 122; c++) {
                    const newWord = word.slice(0, i) + String.fromCharCode(c) + word.slice(i + 1);
                    if (backward.has(newWord)) return level + 1;
                    if (wordSet.has(newWord)) {
                        nextLevel.add(newWord);
                        wordSet.delete(newWord);
                    }
                }
            }
        }
        forward = nextLevel;
        level++;
    }
    return 0;
}
```

For Word Ladder II (LC 126 — return all shortest paths), use BFS to find distance, then backtrack from endWord to beginWord following the BFS graph.

Q5: Explain Dijkstra's algorithm and when it fails.

Answer: Dijkstra's algorithm finds the shortest path from a source node to all other nodes in a weighted graph with **non-negative edge weights**.

Algorithm: Maintain a min-heap (priority queue) of (distance, node) pairs. Start with (0, source). Pop the minimum distance node, relax all its neighbors (if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$ and push to heap). Skip stale entries (when popped distance $>$ current $\text{dist}[u]$).

Time: $O((V + E) \log V)$ with binary heap. **Space:** $O(V)$.

Why it works: The greedy choice — always processing the unvisited node with minimum distance — is optimal because non-negative weights ensure that once a node is extracted from the heap, its distance is final. Adding a longer path can never improve it.

When it fails — negative edge weights: Consider graph: $A \rightarrow B$ (weight 1), $A \rightarrow C$ (weight 2), $C \rightarrow B$ (weight -2). Dijkstra processes B with distance 1 first. But the actual shortest path $A \rightarrow C \rightarrow B$ has distance 0. Dijkstra never revisits B.

Alternatives for negative weights:

- **Bellman-Ford:** $O(V \times E)$, handles negative weights, detects negative cycles. Use for sparse graphs with negative edges.
- **Floyd-Warshall:** $O(V^3)$, all-pairs shortest path. Use when you need distances between all pairs and $V \leq 400$.

Optimizations: A^* adds heuristic for faster search in pathfinding. Bidirectional Dijkstra for single-pair shortest path. For graphs with small integer weights (0-10), use 0-1 BFS or Dial's algorithm (bucket queue) for $O(V + E)$.

Q6: What is digit DP and when would you use it?

Answer: Digit DP counts or finds numbers in a range $[L, R]$ that satisfy digit-based constraints (digit sum, divisibility, no repeated digits, etc.). Brute force checking each number is $O(R - L)$, which fails for ranges up to 10^{18} .

Core idea: Process digits left to right, building the number digit by digit. State: `dp(pos, state, tight, leadingZero)` where:

- `pos` = current digit position
- `state` = problem-specific state (digit sum, remainder, bitmask of used digits)
- `tight` = are we still bounded by the upper limit digit?
- `leadingZero` = are we placing leading zeros?

Template approach:

1. Convert R to string for digit access
2. Use memoization on (pos, state, tight, leadingZero)
3. At each position, try digits 0-9 (limited by tight constraint)
4. Count valid numbers from this state

Example — Count numbers with digit sum = target (LC 902):

```
function count(s: string, minSum: number, maxSum: number): number {
  const memo = new Map<string, number>();
  function dp(pos: number, sum: number, tight: boolean): number {
    if (pos === s.length) return sum >= minSum && sum <= maxSum ? 1 : 0;
    const key = `${pos},${sum},${tight}`;
    if (memo.has(key)) return memo.get(key)!;
    const limit = tight ? +s[pos] : 9;
    let ans = 0;
    for (let d = 0; d <= limit; d++)
      ans += dp(pos + 1, sum + d, tight && d === limit);
    memo.set(key, ans);
    return ans;
  }
  return dp(0, 0, true);
}
```

Answer for range [L, R]: `count(R) - count(L - 1)`.

When to recognize: "Count numbers in range [L, R] where..." with constraints on digits.

Constraints often include digit sum, divisibility, no consecutive same digits, specific digit patterns.

Time complexity is $O(\log(R) \times \text{state_space} \times 10)$.

Q7: How do you find the longest increasing path in a matrix (LC 329)?

Answer: LC 329 asks for the longest increasing path in an $m \times n$ matrix where you can move in four directions and must move to a strictly greater value.

Approach — DFS with memoization (top-down DP): From each cell, DFS in four directions to find the longest path starting from that cell. Memoize results to avoid recomputation. Since we only move to strictly greater values, there are no cycles, so memoization is safe.

```
function longestIncreasingPath(matrix: number[][]): number {
  const m = matrix.length, n = matrix[0].length;
  const memo = new Array(m).fill(0).map(() => new Array(n).fill(0));
  const dirs = [[0, 1], [0, -1], [1, 0], [-1, 0]];

  function dfs(i: number, j: number): number {
    if (memo[i][j] > 0) return memo[i][j];
    memo[i][j] = 1;
    for (const [di, dj] of dirs) {
      const ni = i + di, nj = j + dj;
      if (ni >= 0 && ni < m && nj >= 0 && nj < n && matrix[ni][nj] > matrix[i][j])
        memo[i][j] = Math.max(memo[i][j], 1 + dfs(ni, nj));
    }
    return memo[i][j];
  }

  let maxLen = 0;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++)
      maxLen = Math.max(maxLen, dfs(i, j));
  return maxLen;
}
```

Time: $O(m \times n)$ — each cell computed once. **Space:** $O(m \times n)$ for memo.

Alternative — Topological sort (BFS): Treat as DAG (edges from smaller to larger). Compute in-degrees, process nodes with in-degree 0 in BFS layers. Number of layers = longest path. Same complexity but more complex to implement.

Why not simple DFS without memo? Without memo, visiting cell (i,j) from different paths recomputes the same subproblem, leading to $O(2^{(m+n)})$ exponential time.

Q8: Explain the difference between Kruskal's and Prim's algorithms for MST.

Answer: Both find the Minimum Spanning Tree (MST) — a tree connecting all vertices with minimum total edge weight — but use different approaches.

Kruskal's algorithm is edge-centric. Sort all edges by weight. Iterate through sorted edges, adding each edge if it doesn't create a cycle (checked via Union-Find). Stop when we have $V-1$ edges. Think of it as building the MST by merging connected components.

Time: $O(E \log E)$ dominated by sorting. **Space:** $O(V)$ for Union-Find. **Best for:** Sparse graphs ($E \approx V$) because it only processes edges in sorted order.

Prim's algorithm is vertex-centric. Start from any vertex, maintain a min-heap of edges to unvisited vertices. Repeatedly add the minimum-weight edge connecting a visited vertex to an unvisited one. Think of it as growing the MST one vertex at a time.

Time: $O(E \log V)$ with binary heap. **Space:** $O(V + E)$. **Best for:** Dense graphs ($E \approx V^2$) because it processes vertices, not all edges.

Key difference in practice: For sparse graphs (most real-world networks), Kruskal's is simpler and faster. For dense graphs or when the graph is implicit (like grid problems LC 1584), Prim's avoids sorting all edges.

Both produce the same MST when edge weights are distinct. With equal weights, different MSTs may exist but with the same total cost. In interviews, mention Union-Find for Kruskal's and min-heap for Prim's — these are the supporting data structures that make each algorithm work.

Q9: How would you solve the Regular Expression Matching problem (LC 10)?

Answer: LC 10 matches string s against pattern p where $.$ matches any single character and $*$ matches zero or more of the preceding element. This is a classic 2D DP problem.

State: $dp[i][j]$ = true if $s[0..i-1]$ matches $p[0..j-1]$.

Base case: $dp[0][0] = \text{true}$ (empty matches empty).

Transitions for $dp[i][j]$:

1. If $p[j-1]$ is a letter or $.$:

- Match if characters equal (or $.$) AND $dp[i-1][j-1]$ is true
- $dp[i][j] = \text{match}(s[i-1], p[j-1]) \ \&\& \ dp[i-1][j-1]$

2. If $p[j-1]$ is $*$:

- Zero occurrences: $dp[i][j] = dp[i][j-2]$ (skip char and $*$)
- One or more: $dp[i][j] = \text{match}(s[i-1], p[j-2]) \ \&\& \ dp[i-1][j]$ (current char matches, try matching more)

```
function isMatch(s: string, p: string): boolean {
  const m = s.length, n = p.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(false));
  dp[0][0] = true;

  // Handle patterns like a*, a*b*, a*b*c* matching empty string
  for (let j = 2; j <= n; j += 2) {
    if (p[j - 1] === '*') dp[0][j] = dp[0][j - 2];
  }

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (p[j - 1] === '*') {
        dp[i][j] = dp[i][j - 2]; // Zero occurrences
        if (match(s[i - 1], p[j - 2])) {
          dp[i][j] = dp[i][j] || dp[i - 1][j]; // One or more
        }
      } else {
        dp[i][j] = match(s[i - 1], p[j - 1]) && dp[i - 1][j - 1];
      }
    }
  }
  return dp[m][n];

  function match(sChar: string, pChar: string): boolean {
    return pChar === '.' || sChar === pChar;
  }
}
```

Time: $O(m \times n)$. **Space:** $O(m \times n)$, optimizable to $O(n)$.

This pattern extends to Wildcard Matching (LC 44, '?' and '*') and is a FAANG favorite for testing DP understanding.

Q10: What is tree DP and how does it differ from regular DP?

Answer: Tree DP applies dynamic programming on tree structures, where states are defined on tree nodes and transitions follow parent-child relationships. Unlike linear DP (arrays) or grid DP (matrices), tree DP processes nodes in post-order (children before parent).

Key pattern: For each node, compute answer based on results from child subtrees. Common state designs:

- Include/exclude (House Robber III): rob current node or not

- Max path through node (LC 124): max path sum including this node as turning point
- Minimum coverage (LC 968): min cameras to cover all nodes

Example — Binary Tree Maximum Path Sum (LC 124):

For each node, the max path sum through this node (as a "bridge" connecting left and right subtrees) is: `node.val + max(0, leftGain) + max(0, rightGain)`. The global answer is the max of all such values across all nodes.

```
function maxPathSum(root: TreeNode | null): number {
  let globalMax = -Infinity;

  function maxGain(node: TreeNode | null): number {
    if (!node) return 0;
    const leftGain = Math.max(maxGain(node.left), 0);
    const rightGain = Math.max(maxGain(node.right), 0);
    globalMax = Math.max(globalMax, node.val + leftGain + rightGain);
    return node.val + Math.max(leftGain, rightGain);
  }

  maxGain(root);
  return globalMax;
}
```

Time: $O(n)$ — visit each node once. **Space:** $O(h)$ — recursion stack height.

Difference from regular DP: No explicit DP table — recursion with return values acts as the DP. Processing order is post-order (bottom-up on tree). States are per-node, not per-index. Tree structure prevents cycles, so no cycle detection needed (unlike graph DP).

Q11: How do you solve the Merge k Sorted Lists problem (LC 23)?

Answer: LC 23 merges k sorted linked lists into one sorted list. Three approaches with different trade-offs.

Approach 1 — Min Heap (optimal): Push the head of each list into a min heap. Repeatedly pop the minimum, add to result, push the popped node's next. k elements in heap at any time.

Time: $O(n \log k)$ where n = total nodes, k = number of lists. **Space:** $O(k)$.

```
function mergeKLists(lists: (ListNode | null)[]): ListNode | null {
  const dummy = new ListNode(0);
  let current = dummy;
  const heap: ListNode[] = [];

  for (const list of lists) {
    if (list) heap.push(list);
  }
  heap.sort((a, b) => a.val - b.val);

  while (heap.length > 0) {
    const node = heap.shift()!;
    current.next = node;
    current = current.next;
    if (node.next) {
      heap.push(node.next);
      heap.sort((a, b) => a.val - b.val);
    }
  }
  return dummy.next;
}
```

Approach 2 — Divide and Conquer: Pair up lists and merge pairwise (like merge sort). $\log k$ levels, each level processes all n nodes.

Time: $O(n \log k)$. **Space:** $O(1)$ if merging in-place.

Approach 3 — Brute Force: Collect all values, sort, build new list. $O(n \log n)$ — not optimal but acceptable if k is large.

In interviews: Start with heap approach (shows you know priority queues). Mention divide-and-conquer as alternative. For follow-up "what if lists are arrays?" — same heap approach. "What if data doesn't fit in memory?" — external merge sort with k -way merge from disk.

Q12: Explain the sliding window technique for hard problems.

Answer: Sliding window maintains a window [left, right] over an array/string, expanding right and shrinking left based on constraints. Hard problems add complexity through constraint tracking and window validity.

Template for variable-size window:

```
function slidingWindow(s: string): number {
  let left = 0;
  let result = 0;
  const window = new Map<string, number>(); // Track window state

  for (let right = 0; right < s.length; right++) {
    // Expand: add s[right] to window
    window.set(s[right], (window.get(s[right]) || 0) + 1);

    // Shrink: while window is invalid
    while (!isValid(window)) {
      window.set(s[left], window.get(s[left])! - 1);
      if (window.get(s[left]) === 0) window.delete(s[left]);
      left++;
    }

    // Update result
    result = Math.max(result, right - left + 1);
  }
  return result;
}
```

Hard problem applications:

1. **Minimum Window Substring (LC 76):** Window must contain all characters of t. Track required vs current char counts. Expand until valid, then shrink to find minimum. $O(n)$ time.
2. **Substring with Concatenation of All Words (LC 30):** Fixed window of word length $\times$ num words. Slide one character at a time, check if window contains all words (using hash map comparison).
3. **Longest Substring with At Most K Distinct Characters (LC 340):** Variable window, shrink when distinct count $> k$. Track char frequencies.
4. **Max Consecutive Ones III (LC 1004):** Window can contain at most k zeros. Shrink when zero count exceeds k.

Key insight for hard problems: The validity check function `isValid()` is where problem-specific logic lives. For minimum window, update result during shrink phase (when window is valid). For maximum window, update during expand phase.

Time: $O(n)$ for all — each element enters and leaves window at most once. **Space:** $O(k)$ where k = character set or constraint size.

Q13: How would you approach the N-Queens problem (LC 51)?

Answer: N-Queens places n queens on an $n \times n$ board so no two queens attack each other (same row, column, or diagonal). This is a classic backtracking problem.

Key insight for pruning: Track occupied columns and diagonals using sets, avoiding $O(n)$ conflict checks per placement.

```
function solveNQueens(n: number): string[][] {
  const result: string[][] = [];
  const board = Array.from({ length: n }, () => Array(n).fill('.'));
  const cols = new Set<number>();
  const diag1 = new Set<number>(); // row - col (same diagonal)
  const diag2 = new Set<number>(); // row + col (same anti-diagonal)

  function backtrack(row: number): void {
    if (row === n) {
      result.push(board.map(r => r.join('')));
      return;
    }

    for (let col = 0; col < n; col++) {
      if (cols.has(col) || diag1.has(row - col) || diag2.has(row + col)) continue;

      board[row][col] = 'Q';
      cols.add(col);
      diag1.add(row - col);
      diag2.add(row + col);

      backtrack(row + 1);

      board[row][col] = '.';
      cols.delete(col);
      diag1.delete(row - col);
      diag2.delete(row + col);
    }
  }

  backtrack(0);
  return result;
}
```

Time: $O(n!)$ — try n columns in row 0, $n-1$ in row 1, etc. With pruning, much faster in practice.

Space: $O(n^2)$ for board + $O(n)$ for sets.

Optimization for count only (LC 52): Same backtracking but just count solutions, no board storage. Can also use bitmask DP for larger n .

Follow-up questions: "What about N-Rooks (no diagonal constraint)?" — simpler, just track columns. "Can you solve iteratively?" — Yes, but backtracking is natural. "What's the time for $n=9$?" — 352 solutions, instant with pruning.

Q14: Explain interval DP with the Burst Balloons example (LC 312).

Answer: Interval DP solves problems where the answer for a range $[i, j]$ depends on answers for sub-ranges. The key is determining the order of operations — which element to process last in the range.

Burst Balloons (LC 312): Given balloons with values, burst them to maximize coins. Coins for bursting balloon $i = \text{nums}[i-1] \times \text{nums}[i] \times \text{nums}[i+1]$ (with boundary 1s added). Find maximum coins.

Key insight: Instead of thinking "which balloon to burst first" (hard — bursting changes neighbors), think "which balloon to burst LAST in range $[i, j]$." If balloon k is burst last in range $[i, j]$, its neighbors are $\text{nums}[i-1]$ and $\text{nums}[j+1]$ (unchanged by other bursts in the range).

State: $\text{dp}[i][j]$ = max coins from bursting all balloons in open interval (i, j) .

Transition: $\text{dp}[i][j] = \max(\text{dp}[i][k-1] + \text{dp}[k+1][j] + \text{nums}[i-1] \times \text{nums}[k] \times \text{nums}[j+1])$ for all k in (i, j) .

```
function maxCoins(nums: number[]): number {
  const arr = [1, ...nums, 1];
  const n = arr.length;
  const dp = Array.from({ length: n }, () => new Array(n).fill(0));

  for (let len = 3; len <= n; len++) { // Interval length
    for (let i = 0; i <= n - len; i++) {
      const j = i + len - 1;
      for (let k = i + 1; k < j; k++) {
        dp[i][j] = Math.max(
          dp[i][j],
          dp[i][k] + dp[k][j] + arr[i] * arr[k] * arr[j]
        );
      }
    }
  }
  return dp[0][n - 1];
}
```

Time: $O(n^3)$. **Space:** $O(n^2)$.

Other interval DP problems: LC 664 (Strange Printer), LC 1039 (Min Score Triangulation), LC 375 (Guess Number Higher or Lower II), LC 546 (Remove Boxes). The pattern: iterate by interval length, try all split points.

Q15: How do you solve the Trapping Rain Water problem (LC 42)?

Answer: Trapping Rain Water computes how much water can be trapped between bars in an elevation map. Three approaches from brute force to optimal.

Approach 1 — Two Pointers (optimal):

Key insight: water at position $i = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$. Instead of precomputing maxLeft and maxRight arrays, use two pointers from both ends, tracking maxLeft and maxRight . Move the pointer with smaller max (the bottleneck determines water level).

```
function trap(height: number[]): number {
  let left = 0, right = height.length - 1;
  let maxLeft = 0, maxRight = 0;
  let water = 0;

  while (left < right) {
    if (height[left] <= height[right]) {
      maxLeft = Math.max(maxLeft, height[left]);
      water += maxLeft - height[left];
      left++;
    } else {
      maxRight = Math.max(maxRight, height[right]);
      water += maxRight - height[right];
      right--;
    }
  }
  return water;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Approach 2 — Monotonic Stack: Maintain decreasing stack. When current bar is taller than stack top, water is trapped in the bounded area. Calculate: $\text{width} \times \min(\text{top two stack bars}) - \text{sum of bars in between}$.

Approach 3 — Prefix/Suffix Max arrays: Precompute $\text{maxLeft}[i]$ and $\text{maxRight}[i]$ for each position. $\text{Water}[i] = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$. $O(n)$ time, $O(n)$ space.

In interviews: Start with prefix/suffix (shows understanding), optimize to two pointers (shows optimization skill). For follow-up "what if bars can be moved?" — different problem. "3D trapping rain water (LC 407)?" — min heap approach.

Q16: What is the Union-Find time complexity and why is it nearly $O(1)$?

Answer: Union-Find (Disjoint Set Union) with path compression and union by rank achieves $O(\alpha(n))$ amortized time per operation, where α is the inverse Ackermann function.

$\alpha(n)$ grows so slowly that it's effectively constant for any practical n . $\alpha(10^{80}) \approx 4$. For all inputs that fit in the universe, $\alpha(n) < 5$.

Path compression: During $\text{find}(x)$, make every node on the path point directly to the root. Flattens the tree, making future finds faster.

Union by rank: When merging, attach the shorter tree under the root of the taller tree. Keeps tree height logarithmic without compression, and near-constant with compression.

Without optimizations: $O(n)$ per operation (degenerate linked list). **With path compression only:** $O(\log n)$ amortized. **With both:** $O(\alpha(n))$ amortized.

Proof sketch: Path compression reduces tree height. Union by rank ensures rank (approximate log of subtree size) increases slowly. Together, the amortized cost of m operations on n elements is $O(m \times \alpha(n))$.

Applications beyond connectivity:

- Kruskal's MST (detect cycles while adding edges)
- Number of islands (LC 200, dynamic connectivity)
- Accounts merge (LC 721)
- Smallest string with swaps (LC 1202)
- Minimize malware spread (LC 924)

In interviews, always mention both optimizations. Write the optimized template. State complexity as $O(\alpha(n)) \approx O(1)$ amortized.

Q17: How would you solve the Serialize and Deserialize Binary Tree problem (LC 297)?

Answer: This problem requires converting a binary tree to a string and back. The key is choosing a format that unambiguously represents the tree structure.

Approach 1 — BFS (level-order):

Serialize using BFS with 'null' markers for missing children. Deserialize by reading tokens level-by-level, assigning children to parent nodes in queue.

```

function serialize(root: TreeNode | null): string {
  if (!root) return 'null';
  const result: string[] = [];
  const queue = [root];

  while (queue.length > 0) {
    const node = queue.shift()!;
    if (node) {
      result.push(String(node.val));
      queue.push(node.left!, node.right!);
    } else {
      result.push('null');
    }
  }
  return result.join(',');
}

function deserialize(data: string): TreeNode | null {
  if (data === 'null') return null;
  const nodes = data.split(',');
  const root = new TreeNode(parseInt(nodes[0]));
  const queue = [root];
  let i = 1;

  while (queue.length > 0 && i < nodes.length) {
    const node = queue.shift()!;
    if (nodes[i] !== 'null') {
      node.left = new TreeNode(parseInt(nodes[i]));
      queue.push(node.left);
    }
    i++;
    if (i < nodes.length && nodes[i] !== 'null') {
      node.right = new TreeNode(parseInt(nodes[i]));
      queue.push(node.right);
    }
    i++;
  }
  return root;
}

```

Approach 2 — Preorder with null markers: More space-efficient for sparse trees. Serialize:

`root.val, serialize(left), serialize(right)` , using 'null' for null nodes. Deserialize: read value, recursively build left and right.

Time: $O(n)$ for both. **Space:** $O(n)$ for serialized string.

Follow-ups: "What about N-ary tree?" — BFS with all children listed per node. "Can you compress the encoding?" — Use binary format or run-length encoding on nulls. "What if tree is complete?" — Array representation without nulls (heap-style indexing).

Q18: Explain the difference between backtracking and DP. When do you choose each?

Answer: Both solve optimization and enumeration problems, but differ in how they handle overlapping subproblems.

Backtracking explores all possibilities using recursion, pruning branches that can't lead to valid solutions. It builds solutions incrementally and undoes choices (backtracks) when a path fails. Best for: finding ALL solutions (N-Queens, Sudoku, permutations/combinations), constraint satisfaction, problems where the search space is pruned heavily.

Dynamic Programming stores results of subproblems to avoid recomputation. Best for: optimization problems (min/max/count), problems with overlapping subproblems and optimal substructure, when the state space is polynomial.

Decision framework:

Signal	Choose
"Find all solutions"	Backtracking
"Find minimum/maximum/count"	DP
"Return any one solution"	Either (backtrack may be faster with pruning)
Overlapping subproblems exist	DP
State space is exponential with heavy pruning	Backtracking
State space is polynomial	DP

Example — Word Break:

- "Return true if s can be segmented" → DP (boolean, overlapping subproblems)
- "Return all possible sentences" → Backtracking with DP memo (Word Break II, LC 140)

Combination: Many hard problems combine both. Backtracking to build solutions, with memoization to avoid revisiting states (Word Break II, Palindrome Partitioning II). This is sometimes called "DP with backtracking" or "memoized backtracking."

Time complexity: Backtracking is often $O(k \times n!)$ or $O(k \times 2^n)$ where k = number of solutions. DP is polynomial in state space size.

Q19: How do you solve the Median of Two Sorted Arrays problem (LC 4)?

Answer: LC 4 finds the median of two sorted arrays in $O(\log(\min(m,n)))$ time — one of the hardest binary search problems.

Key insight: The median partition divides both arrays such that all elements in the left half are $\leq$ all elements in the right half. If we take i elements from `nums1` and j elements from `nums2` where $i + j = (m + n + 1) / 2$, the median is determined by $\max(\text{left1}, \text{left2})$ and $\min(\text{right1}, \text{right2})$.

Binary search on the smaller array: For each partition point i in `nums1`, $j = (m + n + 1) / 2 - i$ in `nums2`. Check if $\max(\text{nums1}[i-1], \text{nums2}[j-1]) \leq \min(\text{nums1}[i], \text{nums2}[j])$. If not, adjust i via binary search.

```
function findMedianSortedArrays(nums1: number[], nums2: number[]): number {
  if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);

  const m = nums1.length, n = nums2.length;
  let lo = 0, hi = m;

  while (lo <= hi) {
    const i = Math.floor((lo + hi) / 2);
    const j = Math.floor((m + n + 1) / 2) - i;

    const maxLeft1 = i === 0 ? -Infinity : nums1[i - 1];
    const minRight1 = i === m ? Infinity : nums1[i];
    const maxLeft2 = j === 0 ? -Infinity : nums2[j - 1];
    const minRight2 = j === n ? Infinity : nums2[j];

    if (maxLeft1 <= minRight2 && maxLeft2 <= minRight1) {
      if ((m + n) % 2 === 0) {
        return (Math.max(maxLeft1, maxLeft2) + Math.min(minRight1, minRight2)) / 2;
      }
      return Math.max(maxLeft1, maxLeft2);
    } else if (maxLeft1 > minRight2) {
      hi = i - 1;
    } else {
      lo = i + 1;
    }
  }
  return 0;
}
```

Time: $O(\log(\min(m,n)))$. **Space:** $O(1)$.

Why binary search on smaller array: Ensures j is always valid (non-negative and $\leq n$). The partition approach elegantly handles both odd and even total lengths.

Q20: What strategies do you use for solving hard DP problems in interviews?

Answer: Hard DP problems follow recognizable patterns. My systematic approach:

Step 1 — Identify the pattern:

- Linear DP (1D): $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$, etc.
- Grid DP (2D): $dp[i][j]$ depends on adjacent cells
- Interval DP: $dp[i][j]$ for range $[i, j]$, iterate by length
- Tree DP: post-order computation on nodes
- Bitmask DP: $dp[mask]$ for subset states
- Digit DP: $dp[pos][state]$ for number construction
- State machine DP: $dp[day][state]$ for stock problems

Step 2 — Define state precisely: What information do I need to make a decision? " $dp[i] = \text{max profit considering first } i \text{ elements}$ " is better than " $dp[i] = \text{something about } i$."

Step 3 — Write the recurrence: Express $dp[state]$ in terms of smaller states. This is the hardest part — draw examples, work through small cases manually.

Step 4 — Identify base cases: $dp[0] = ?$, $dp[1] = ?$

Step 5 — Determine iteration order: Ensure dependencies are computed before they're needed. Interval DP iterates by length; bitmask DP iterates by mask size.

Step 6 — Optimize space if needed: If $dp[i]$ only depends on $dp[i-1]$ and $dp[i-2]$, use two variables instead of an array.

Common hard DP patterns at 80 LPA:

- **Stock problems (LC 123, 188):** State = (day, transactions_remaining, holding). Generalize with k transactions.
- **String matching (LC 10, 44, 72):** 2D DP with char-by-char comparison and special pattern handling.
- **Interval (LC 312, 664):** Try all split points k in range $[i, j]$.
- **Bitmask (LC 847, 1125):** Iterate masks, try adding each unused element.
- **Digit (LC 902, 233):** Process digit by digit with tight/loose bound.

Time management: Spend 5 minutes identifying the pattern. If stuck after 10 minutes on recurrence, discuss approach with interviewer — partial credit for correct state definition.

11. Where to Practice

Platforms

- [LeetCode](#) — Filter by Hard, sort by acceptance rate
- [NeetCode 150](#) — Curated list with video solutions
- [Codeforces](#) — Div 2 C/D for contest practice
- [AtCoder](#) — DP and graph contests

Hard Problem Lists

- [LeetCode Hard List \(Top 100 Liked\)](#)
- [NeetCode All Problems](#)
- [Grind 75 + Hard Extension](#)

Books

- "Cracking the Coding Interview" — Hard problem selection
- "Competitive Programming 3" by Halim — Advanced algorithms
- "Algorithm Design Manual" by Skiena — Pattern reference

YouTube Channels

- [NeetCode](#) — Hard problem walkthroughs
- [William Lin \(tmwilliamlin168\)](#) — LeetCode contest solutions
- [Abhinav Rahul](#) — Hard DP and graph problems

Weekly Practice Plan (Hard Focus)

Day	Focus	Problems
Mon	Graph (Dijkstra, Topo, MST)	2-3 hard
Tue	Advanced DP (bitmask, digit, interval)	2-3 hard
Wed	Monotonic Stack/Deque	2 hard
Thu	Segment Tree / Trie	2 hard
Fri	Mixed Hard (timed, 45 min each)	2 hard
Sat	Mock interview (1 hard + follow-ups)	1-2 hard
Sun	Review failed problems, re-solve	Review

Key Milestones for 80 LPA

- Solve 100+ hard LeetCode problems independently
- Complete NeetCode 150 hard subset (40+ problems)
- Solve 2 hard problems in 45-minute mock interviews
- Explain time/space complexity and alternative approaches for every solution
- Recognize patterns within 5 minutes of reading problem

Last updated: August 2026 | Target: 80+ LPA SDE (Google L5, Meta E4, Amazon SDE3, Microsoft L63+)

Advanced Data Structures and Algorithms Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers hard LeetCode patterns, advanced dynamic programming, and graph algorithms for senior software engineering interviews at product companies. Spell out abbreviations on first use.

1. What is dynamic programming (DP)?

Dynamic Programming (DP) is an optimization technique that solves complex problems by breaking them into overlapping subproblems and storing results to avoid recomputation. It applies when a problem has optimal substructure (optimal solution built from optimal sub-solutions) and overlapping subproblems. Reduces exponential brute force to polynomial time. Classic examples: Fibonacci, knapsack, longest common subsequence.

2. How do you identify a dynamic programming problem?

Look for: (1) optimization (minimum, maximum, count ways), (2) choices at each step affecting future steps, (3) recursive structure with repeated subproblems. Ask: "Can I define a recurrence relation?" and "Do subproblems repeat?" If yes to both, try Dynamic Programming. Keywords: "minimum cost," "maximum profit," "number of ways," "longest/shortest."

3. What is the difference between memoization and tabulation?

Memoization (top-down): start with recursive solution, cache results in a hash map or array as you compute—lazy, only computes needed subproblems. Tabulation (bottom-up): fill a table iteratively from base cases up—no recursion stack, often better space optimization. Both achieve the same time complexity. Prefer memoization for intuitive thinking; tabulation for space-optimized solutions.

4. Explain the 0/1 knapsack problem?

Given items with weights and values, and a knapsack capacity, maximize total value without exceeding capacity. Each item can be taken at most once (0 or 1). Recurrence: `dp[i][w] =`

$\max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i])$ if $weight[i] \leq w$. Time: $O(n \times capacity)$. Space optimize to $O(capacity)$ using 1D array iterated backwards. Foundation for resource allocation problems.

5. Explain the unbounded knapsack problem?

Same as 0/1 knapsack but each item can be used unlimited times. Recurrence: $dp[w] = \max(dp[w], dp[w-weight[i]] + value[i])$ for all items. Iterate forward (not backward) when space-optimized. Classic application: coin change (minimum coins to make amount), rod cutting. Differs from 0/1 only in the inner loop direction for space optimization.

6. What is the longest common subsequence (LCS)?

Given two strings, find the longest subsequence (characters in order, not necessarily contiguous) common to both. Recurrence: if chars match, $dp[i][j] = dp[i-1][j-1] + 1$; else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. Time: $O(m \times n)$. Used in diff tools, bioinformatics (DNA alignment), and as building block for edit distance and shortest common supersequence.

7. What is the longest increasing subsequence (LIS)?

Find the longest subsequence where elements are strictly increasing. $O(n^2)$ Dynamic Programming: $dp[i] = \max(dp[j] + 1)$ for all $j < i$ where $nums[j] < nums[i]$. Optimized $O(n \log n)$ using patience sorting with binary search—maintain array of smallest tail values for each length. LeetCode 300. Important for box nesting and Russian doll envelopes.

8. Explain edit distance (Levenshtein distance)?

Minimum number of operations (insert, delete, replace) to transform string A into string B. Recurrence: if chars match, $dp[i][j] = dp[i-1][j-1]$; else $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$. Time: $O(m \times n)$. Used in spell checkers, DNA analysis, and fuzzy matching. LeetCode 72.

9. What is matrix chain multiplication?

Given matrices with dimensions, find the optimal parenthesization to minimize scalar multiplications. $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + cost)$ for all k between i and j .

Interval Dynamic Programming pattern—iterate by length of interval. Time: $O(n^3)$. Teaches interval DP thinking applicable to burst balloons, palindrome partitioning.

10. What is interval dynamic programming?

Interval DP solves problems on ranges $[i, j]$ by combining solutions of smaller sub-intervals. Pattern: iterate by interval length, for each interval try all split points. Examples: matrix chain multiplication, burst balloons (LeetCode 312), palindrome partitioning, strange printer. Key insight: answer for $[i, j]$ depends on answers for $[i, k]$ and $[k+1, j]$.

11. Explain the coin change problem?

Given coin denominations and a target amount, find minimum coins needed (or count ways). Minimum coins (unbounded knapsack): `dp[a] = min(dp[a], dp[a-coin] + 1)`. Count ways: `dp[a] += dp[a-coin]`. Handle impossible case (return -1). LeetCode 322 and 518. Watch for Integer Overflow and use long if needed for large counts.

12. What is bitmask dynamic programming?

Bitmask DP uses an integer bitmask to represent a subset of elements (bit i set = element i included). State: `dp[mask][i]` = best value using subset `mask` ending at i . Used for Traveling Salesperson Problem, assignment problems, and subset enumeration. Requires $n \leq 20$ typically ($2^{20} \approx 1$ million states). Iterate masks in increasing order of set bits.

13. Explain the Traveling Salesperson Problem (TSP) with DP?

Visit every city exactly once, return to start, minimize total distance. Bitmask DP: `dp[mask][i]` = min cost to visit cities in `mask`, ending at city i . Transition: try all previous cities j in `mask`. Time: $O(n^2 \times 2^n)$. Exact solution for small n ($n \leq 15$). NP-hard—no polynomial exact algorithm known. Held-Karp algorithm.

14. What is tree dynamic programming?

Tree DP computes optimal values on trees, typically with post-order traversal (process children before parent). State defined per node: `dp[node]` = best value for subtree rooted at node. Examples: house robber III (LeetCode 337), binary tree maximum path sum (LeetCode 124), tree diameter. Key: combine results from left and right children at each node.

15. Explain the house robber on a binary tree?

Each node has a value; cannot rob adjacent nodes (parent and child). For each node, return pair (rob_this, skip_this). `rob = node.val + left.skip + right.skip ; skip = max(left.rob, left.skip) + max(right.rob, right.skip)` . Post-order traversal. LeetCode 337. Extends linear house robber to tree structure.

16. What is digit dynamic programming?

Digit DP solves counting problems over number ranges by processing digits position by position with constraints. State: (position, tight, leading_zeros, other_flags). "Tight" means current prefix equals the number prefix—limits next digit choices. Examples: count numbers with digit sum = K, numbers without consecutive 1s in binary. Common in competitive programming and hard LeetCode.

17. What is the difference between graphs and trees for algorithms?

A tree is a connected acyclic graph with n nodes and $n-1$ edges, having exactly one path between any two nodes. Graphs may have cycles, multiple paths, and disconnected components. Trees enable simpler algorithms (no cycle detection needed). Graphs require visited tracking and handle more complex structures. Many tree problems extend to graphs with cycle handling.

18. Explain Dijkstra's algorithm?

Dijkstra finds shortest paths from a source to all nodes in a weighted graph with non-negative edges. Uses a min-priority queue: extract closest unvisited node, relax its neighbors. Time: $O((V + E) \log V)$ with binary heap. Does not work with negative edges (use Bellman-Ford). LeetCode applications: network delay time, cheapest flights within K stops.

19. Explain Bellman-Ford algorithm?

Bellman-Ford finds shortest paths allowing negative edge weights. Relax all edges $V-1$ times; V -th iteration detects negative cycles. Time: $O(V \times E)$. Slower than Dijkstra but handles negative weights. Used in currency arbitrage detection and difference constraints. LeetCode: cheapest flights with at most K stops (modified Bellman-Ford).

20. What is Floyd-Warshall algorithm?

Floyd-Warshall computes all-pairs shortest paths in $O(V^3)$. DP recurrence: `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` for each intermediate k . Works with negative edges (no negative cycles). Simple to implement—three nested loops. Used when you need distances between all pairs, not just single source.

21. Explain topological sort?

Topological sort orders nodes in a Directed Acyclic Graph (DAG) such that for every edge $u \rightarrow v$, u comes before v . Algorithms: Kahn's (BFS with in-degree counting) or DFS-based (post-order reverse). Detects cycles (if sort includes fewer than V nodes). Applications: course schedule, build order, task scheduling with dependencies. LeetCode 207, 210.

22. What is Union-Find (Disjoint Set Union)?

Union-Find maintains a partition of elements into disjoint sets with near-constant time union and find operations. Uses parent array and rank/size optimization with path compression. Applications: detect cycles in undirected graphs, count connected components, Kruskal's minimum spanning tree. Template problem: number of provinces, redundant connection.

23. Explain Kruskal's minimum spanning tree algorithm?

Kruskal finds Minimum Spanning Tree (MST)—connects all nodes with minimum total edge weight. Sort edges by weight; add each edge if it does not create a cycle (Union-Find check). Time: $O(E \log E)$. Works well for sparse graphs. Compare with Prim's for dense graphs. LeetCode: connecting cities with minimum cost.

24. Explain Prim's minimum spanning tree algorithm?

Prim builds MST by growing from a start node: maintain min-heap of edges from visited set to unvisited. Extract minimum edge, add node, push new edges. Time: $O(E \log V)$ with binary heap. Better for dense graphs. Both Kruskal and Prim produce same MST weight; choice depends on graph density and implementation convenience.

25. What is a strongly connected component (SCC)?

In a directed graph, an SCC is a maximal set of nodes where every node is reachable from every other node. A directed graph decomposes into SCCs forming a DAG (Directed Acyclic Graph) of components. Applications: social network analysis, compiler optimization (variable dependency), 2-SAT problem. Find with Kosaraju's or Tarjan's algorithm.

26. Explain Kosaraju's algorithm for SCC?

Two-pass Depth-First Search (DFS): (1) DFS on original graph, push nodes to stack on finish. (2) DFS on transposed graph (reverse all edges), pop from stack—each DFS tree is one SCC. Time: $O(V + E)$. Intuitive but requires graph transposition. Tarjan's algorithm achieves same result in one pass.

27. What is bipartite graph checking?

A graph is bipartite if nodes can be colored with two colors such that no adjacent nodes share a color. Check using BFS (Breadth-First Search) or DFS coloring: alternate colors; if conflict found, not bipartite. Equivalent: no odd-length cycles. Applications: matching problems, conflict-free scheduling. LeetCode 785. Also detect using Union-Find with parity.

28. Explain the A* search algorithm?

A* finds shortest path using heuristic function: $f(n) = g(n) + h(n)$, where g is cost from start, h is estimated cost to goal. With admissible heuristic (never overestimates), A* is optimal. Used in game pathfinding, GPS navigation, puzzle solving. More efficient than Dijkstra when good heuristic available. LeetCode: shortest path in grid with obstacles (BFS is simpler alternative).

29. What is backtracking?

Backtracking builds solutions incrementally and abandons partial solutions that cannot lead to valid complete solutions ("prune" the search tree). Template: choose → explore → unchoose. Examples: N-Queens, Sudoku, permutations, combinations, subset generation. Time often exponential but pruning makes it practical. Key: recognize when to undo the last choice.

30. Explain the N-Queens problem?

Place N queens on an $N \times N$ chessboard so no two queens attack each other (same row, column, or diagonal). Backtracking: place queen row by row, check column and diagonal conflicts using sets, recurse to next row, backtrack if stuck. Count or return all solutions. LeetCode 51/52. Optimize diagonal tracking with `row+col` and `row-col` keys.

31. What is sliding window technique?

Sliding window maintains a window [left, right] over an array/string, expanding right and shrinking left based on constraints. Fixed window: size K subarray maximum. Variable window: longest substring without repeating characters. Time: $O(n)$ —each element enters and exits once. One of the most common patterns in medium LeetCode problems.

32. What is two pointers technique?

Two pointers use two indices moving through a structure, often from opposite ends or at different speeds. Patterns: opposite ends (sorted array two sum), same direction (remove duplicates, linked list cycle), fast/slow (cycle detection, find middle). Reduces $O(n^2)$ brute force to $O(n)$. Works on sorted arrays, linked lists, and strings.

33. Explain monotonic stack?

Monotonic stack maintains elements in sorted order (increasing or decreasing). Used to find next greater/smaller element for each position in $O(n)$. Push elements; pop when top violates monotonic property. Applications: largest rectangle in histogram, daily temperatures, trapping rain water. LeetCode 496, 739, 84. One of the highest-yield patterns for hard problems.

34. Explain monotonic queue/deque?

Monotonic deque maintains min/max in a sliding window in $O(n)$. Deque stores indices in decreasing (for max) or increasing (for min) value order. Remove expired front element when window slides. Applications: sliding window maximum (LeetCode 239), constrained subsequence sum. Combines sliding window with deque efficiency.

35. What is trie (prefix tree)?

A trie is a tree where each node represents a character; paths from root to nodes represent strings. Enables $O(L)$ insert/search/prefix check where L is word length. Used for autocomplete, spell checking, word search, and prefix matching. Space-heavy but fast for string collections. LeetCode 208 (implement trie), 212 (word search II).

36. What is segment tree?

Segment tree is a binary tree for range queries (sum, min, max) with $O(\log n)$ query and update. Each node covers a range; children cover halves. Build: $O(n)$; query/update: $O(\log n)$. Used when you need both range queries and point/range updates. Heavier than prefix sums but supports updates. Lazy propagation handles range updates efficiently.

37. What is binary indexed tree (Fenwick tree)?

Fenwick tree (Binary Indexed Tree—BIT) supports prefix sum query and point update in $O(\log n)$ with simpler code than segment tree. Uses array where index i stores sum of range $(i - \text{lowbit}(i), i]$. Less memory than segment tree. Cannot do arbitrary range queries as easily. Good for competitive programming and inversion count problems.

38. Explain the word break problem?

Given a string and dictionary, determine if string can be segmented into dictionary words. DP:

$\text{dp}[i] = \text{true}$ if $\text{dp}[j]$ is true and $s[j:i]$ is in dictionary, for any $j < i$. Time: $O(n^2 \times m)$ where m is word check time (use hash set for $O(1)$ lookup). LeetCode 139. Extension: return all possible sentences (backtracking on DP table).

39. What is the regular expression matching problem?

Match string s against pattern p with $.$ (any char) and $*$ (zero or more of preceding element). DP: $\text{dp}[i][j]$ = does $s[0..i]$ match $p[0..j]$? Handle $*$ by trying zero occurrences or one+ occurrences of preceding char. Time: $O(m \times n)$. LeetCode 10—classic hard DP. Wildcard matching (LeetCode 44) uses similar but different $*$ semantics.

40. Explain maximum flow (Ford-Fulkerson)?

Maximum flow finds the greatest flow from source to sink in a capacity network. Ford-Fulkerson repeatedly finds augmenting paths (BFS $\rightarrow$ Edmonds-Karp) and pushes flow until no path exists. Max-flow min-cut theorem: maximum flow equals minimum cut capacity. Applications: bipartite matching, network reliability. LeetCode 2699 (hard—transform to flow).

41. What is the meeting rooms II problem?

Given meeting intervals, find minimum conference rooms needed. Approach 1: min-heap of end times—sort by start, if earliest ending meeting finished, reuse room. Approach 2: sweep line—sort all start/end events, track concurrent meetings. Time: $O(n \log n)$. LeetCode 253. Tests priority queue and interval processing skills.

42. Explain merge intervals?

Given intervals, merge all overlapping ones. Sort by start time; iterate, if current overlaps with last merged, extend end; else add new interval. Time: $O(n \log n)$. Foundation for interval scheduling problems. Variants: insert interval, employee free time, interval list intersection. Always sort first for interval problems.

43. What is the subarray sum equals K problem?

Count subarrays with sum equal to K. Use prefix sum hash map: for each prefix sum, add count of (prefixSum - K) seen before. Handles negative numbers (sliding window cannot). Time: $O(n)$. LeetCode 560. Important: initialize map with {0: 1} for subarrays starting at index 0. Extension: subarray sum divisible by K.

44. Explain binary search on answer?

When answer lies in a range and you can check "is X feasible?" in polynomial time, binary search on the answer space. Examples: minimum capacity to ship packages in D days, koko eating bananas, split array largest sum. Pattern: lo = min possible, hi = max possible, mid = (lo+hi)/2, if feasible hi=mid else lo=mid+1. Turns optimization into decision problem.

45. What is the trick for cyclic/rotated array search?

Rotated sorted array has one pivot where order breaks. Modified binary search: identify which half is sorted, check if target lies in sorted half. Time: $O(\log n)$. Also: find minimum in rotated sorted array (LeetCode 153). Key insight: at least one half is always sorted in a rotated array.

46. How do you approach hard graph problems in interviews?

Steps: (1) Clarify directed/undirected, weighted/unweighted, cycles. (2) Choose representation (adjacency list for sparse). (3) Pick algorithm: BFS (unweighted shortest path), Dijkstra (non-negative weights), DFS (connectivity, cycles), Union-Find (dynamic connectivity), topological sort (dependencies). (4) Handle edge cases: disconnected, self-loops, empty graph. (5) State time/space complexity.

47. What is space optimization for 2D DP?

When `dp[i][j]` depends only on row $i-1$, use two 1D arrays or a single array updated in reverse (0/1 knapsack). For LCS-style, keep two rows alternating. Reduces $O(m \times n)$ space to $O(\min(m, n))$. Mention this optimization after coding the 2D solution—shows advanced understanding without complicating initial code.

48. How do you practice advanced Data Structures and Algorithms (DSA)?

Focus on patterns, not problem count: 2-3 problems per pattern (DP, graphs, backtracking, binary search on answer). Use LeetCode curated lists and tag filters. After solving, study editorial for alternative approaches. Time yourself (45 min for medium, 60 for hard). Redo failed problems after 1 week. Mock interviews on Pramp or Interviewing.io.

49. What is the difference between Breadth-First Search and Depth-First Search on graphs?

BFS explores level by level using a queue—finds shortest path in unweighted graphs, $O(V + E)$. DFS explores as deep as possible using recursion/stack—detects cycles, topological sort, connected components, $O(V + E)$. Choose BFS for shortest path; DFS for exhaustive search, backtracking, and tree-like traversal. BFS uses $O(V)$ queue space; DFS uses $O(V)$ recursion stack.

50. Summarize advanced DSA interview checklist?

Master: DP patterns (knapsack, LCS, interval, bitmask, tree), graph algorithms (Dijkstra, topological sort, Union-Find, SCC), and techniques (sliding window, monotonic stack, binary search on answer, trie). For hard problems: clarify constraints, start with brute force, optimize with pattern recognition, code cleanly, test edge cases, state complexity. Communicate thought process throughout.